



Click && Build

Generator de site-uri web cu inteligență artificială și
management dinamic al contextului

Secțiunea Web

Documentație tehnică

Leuștean Victor-Constantin

Târîța Alexia-Elena

Colegiul Național „Calistrat Hogaș” Piatra-Neamț

INFOEDUCAȚIE 2026 · ETAPA NAȚIONALĂ

Cuprins

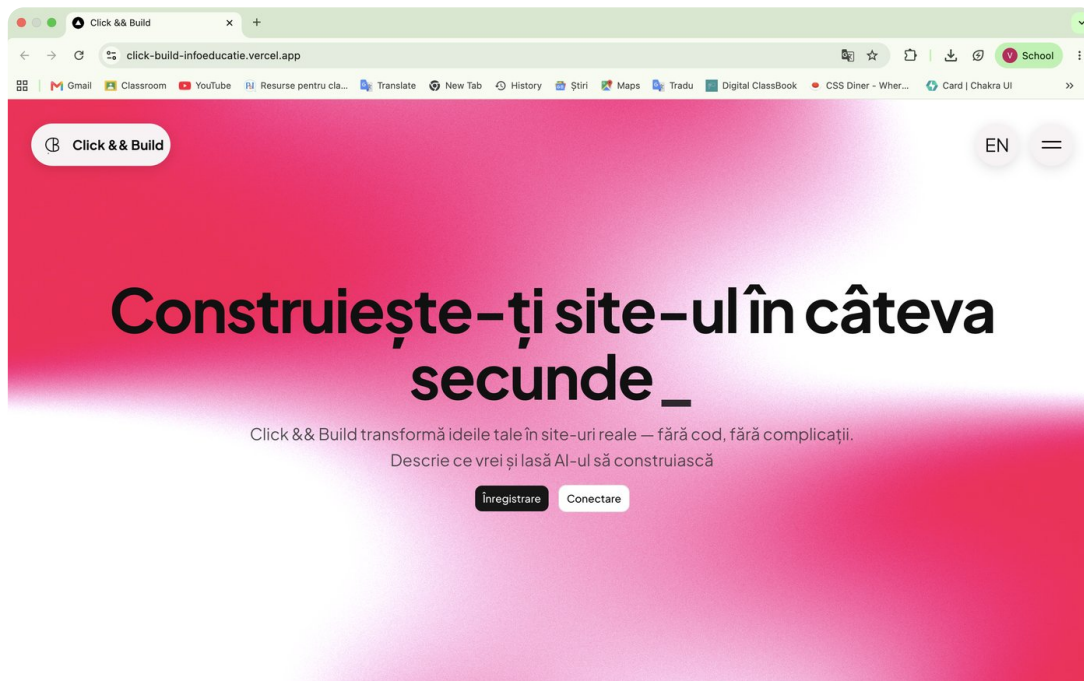
1. Introducere	2
2. Inginerie web și arhitectură	3
2.1 Structurarea codului și versionare	3
2.2 Pipeline-ul de generare	4
2.3 DCS — Delta Context System	5
2.4 Baza de date	6
2.5 Testare	7
3. Funcționalități	8
4. Interacțiuni, design și accesibilitate	9
5. Originalitate	11
6. Securitate	12
7. Performanță și validare cantitativă	13

1. Introducere

1.1 Scopul platformei

Click && Build este o platformă web care generează site-uri complete pornind de la o simplă descriere în limbaj natural. Utilizatorul descrie ce site dorește („un site pentru brutăria mea, cu meniu și formular de contact”), iar aplicația produce un site React funcțional, cu design profesional, conținut realist și imagini reale — pe care apoi îl poate edita conversațional și publica online la o adresă proprie.

Problema adresată: generatoarele de site-uri cu AI existente reîncarcă întreaga conversație la fiecare modificare. Pe măsură ce utilizatorul editează iar și iar, contextul trimis modelului crește necontrolat — costul explodează, iar modelul „se pierde” în istoricul lung și ratează detalii (fenomenul *lost in the middle*). Click && Build rezolvă această problemă printr-un sistem propriu de gestionare a contextului, **DCS**.



Pagina principală (landing) — hero-ul cu titlul animat și butoanele Înregistrare / Conectare.

1.2 Fluxul utilizatorului

- **Describe** — utilizatorul introduce o descriere a site-ului dorit;
- **Sugestii** — aplicația propune un titlu, un public țintă și o paletă de culori;
- **Generare** — un pipeline pe mai multe modele AI produce site-ul complet;
- **Editare** — utilizatorul rafinează site-ul prin mesaje în limbaj natural;
- **Istoric & rollback** — orice versiune anterioară poate fi reconstruită;
- **Publicare** — site-ul devine accesibil public la o adresă proprie.

Ce diferențiază proiectul

Click & Build nu este un simplu „wrapper” peste un model AI. Contribuția centrală este **DCS (Delta Context System)** — un sistem de versionare pe diferențe (delte) combinat cu căutare semantică, care menține contextul trimis modelului **mărginit și relevant**, indiferent de câte editări sunt efectuate.

2. Inginerie web și arhitectură

2.1 Structurarea codului și versionare

Codul este organizat modular, pe responsabilități clare:

- `app/api/` — rute serverless (build, edit, suggestion, publish, versions, rollback);

- `lib/data/` — stratul unic de acces la date, unde fiecare funcție impune filtrarea pe utilizator;
- `lib/preview/` — logica de randare izolată a codului generat;
- `lib/benchmark/` — simularea cantitativă DCS;
- `lib/validation/` — schemele de validare a datelor de intrare;
- `lib/i18n/` — sistemul de internaționalizare;
- `components/` — interfața (editor, preview, landing).

Versionarea codului sursă este gestionată prin **Git**, cu istoric public pe GitHub. Proiectul este deployat în producție pe **Vercel**.

2.2 Pipeline-ul de generare

Generarea unui site nu este un singur apel către un model, ci un **pipeline orchestrat pe mai multe modele specializate**, fiecare cu un rol distinct. Aceasta este o contribuție proprie — coordonarea modelelor și fluxul de date între ele.

prompt utilizator

↓

→ generează cuvinte-cheie pentru imagini

↓

→ caută imagini reale (nu placeholder)

↓

→ alege un design system ca inspirație

↓

→ generează componenta React completă

↓

→ salvează versiunea 0 (STORE) + embedding

Fiecare etapă alimentează etapa următoare. Modelul specializat în cod primește exact ce-i trebuie — imagini reale, direcție de design, culorile alese de utilizator — ceea ce produce site-uri net superioare unui singur apel generic.

Imagini reale, nu generice

Spre deosebire de abordările care folosesc placeholder-uri, pipeline-ul caută imagini reale relevante temei prin cuvinte-cheie generate de AI — un site de brutărie primește fotografii de pâine și patiserie reale.

Design system ca inspirație, nu ca template

Aplicația folosește patru sisteme de design (extrase din site-uri reale) drept **ghid de inspirație** pentru model — tipografie, spacing, geometria componentelor. Acestea **nu** sunt template-uri copiate: culorile și titlul alese de utilizator au prioritate absolută, iar modelul generează cod original inspirat de principiile de design, nu o clonă.

Efort de raționament adaptat task-ului

Generarea inițială (complexă) folosește efort de raționament normal, în timp ce editările (modificări punctuale) folosesc efort redus (`reasoning: low`). Astfel latența editărilor scade, potrivit „gândirea” modelului la complexitatea reală a fiecărui task.

2.3 DCS — Delta Context System

DCS este **contribuția centrală și cea mai originală** a proiectului. Rezolvă problema creșterii necontrolate a contextului în editarea iterativă cu AI.

Problema

Într-o abordare naivă, fiecare editare trimite modelului întreaga conversație: toate instrucțiunile anterioare plus toate versiunile intermediare ale codului. La a 40-a editare, contextul devine enorm — costul crește exploziv, iar modelul suferă de *lost in the middle*: detaliile din mijlocul contextului lung nu mai sunt percepute corect, ceea ce degradează calitatea editărilor.

Soluția: stocare pe delte + retrieval semantic

DCS stochează versiunile site-ului **nu ca fișiere complete, ci ca diferențe (delte)**:

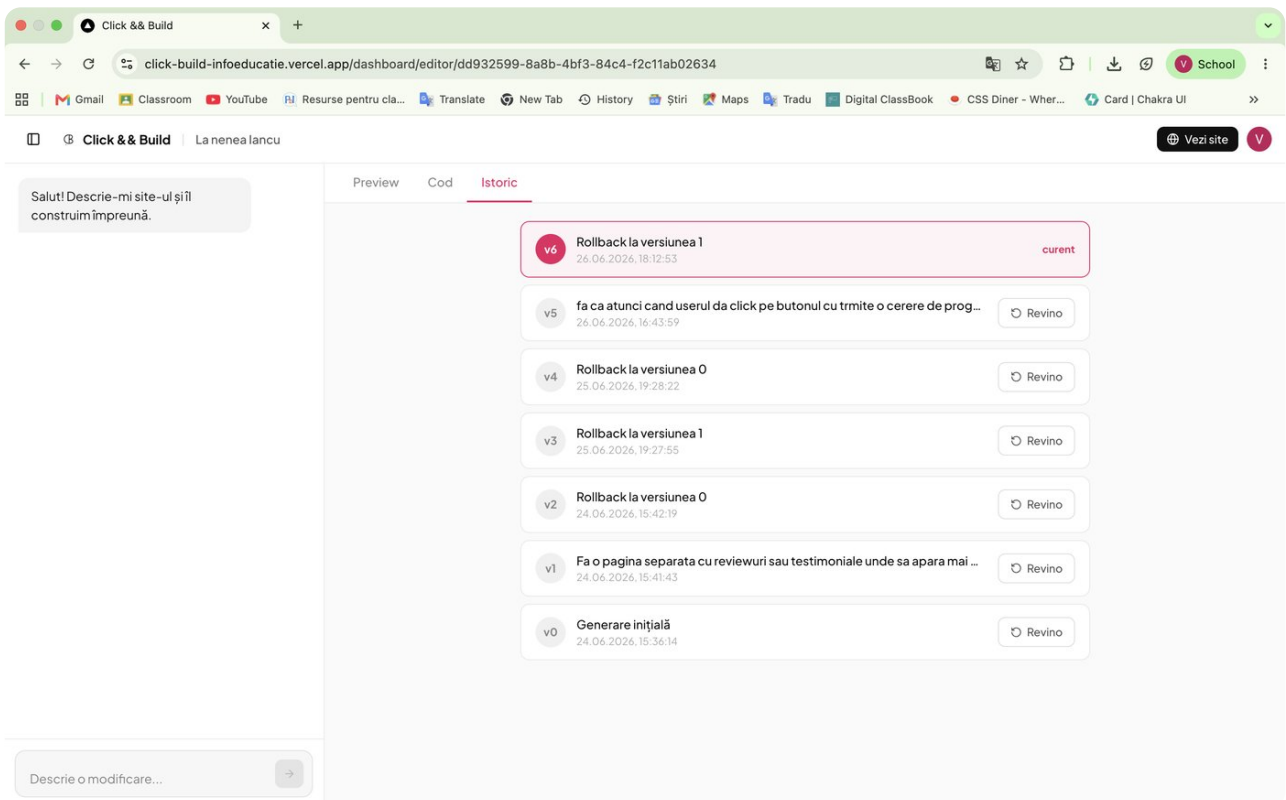
- **Versiunea 0 (STORE)** — codul complet inițial, imutabil;
- **Versiunile 1..N (delta nodes)** — doar diferența (diff) față de versiunea anterioară.

Reconstrucția oricărei versiuni K se face pornind de la STORE și aplicând diff-urile 1..K în ordine. Astfel rollback-ul devine posibil ca efect secundar al designului, fără a dubla stocarea.

Retrieval semantic prin embeddings

Fiecare instrucțiune de editare este codată ca un **vector de 1536 de dimensiuni** (embedding) care îi reprezintă înțelesul semantic, stocat în baza de date vectorială (pgvector) cu index HNSW. La o editare nouă:

- instrucțiunea nouă este codată ca vector;
- se caută semantic, prin **similaritate cosine**, cele mai relevante 5 instrucțiuni anterioare — după *sens*, nu după cuvinte („fă bara albastră” se potrivește cu „schimbă culoarea navbar-ului în navy” deși n-au cuvinte comune);
- în context se injectează **codul curent + doar acele 5 instrucțiuni relevante**, nu tot istoricul.



Timeline-ul de versiuni DCS (tab-ul „Istoric”) — fiecare versiune cu instrucțiunea ei și butonul „Revino” pentru rollback semantic.

Efectul: context mărginit și relevant

Contextul trimis modelului are două componente: **codul curent** (care crește lent, cu complexitatea site-ului — necesar oricum) și **K instrucțiuni relevante** (mereu mărginit la 5). Componenta care în abordarea naivă crește exploziv — istoricul — este eliminată. Rezultatul: mai puțini tokeni, cost redus și relevanță mai bună a editărilor.

2.4 Baza de date

Aplicația folosește **PostgreSQL** (prin Supabase) cu extensia **pgvector** pentru căutarea vectorială. Schema este proiectată în jurul DCS:

Tabel	Rol
<code>sites</code>	Site-urile utilizatorilor: cod curent, versiune curentă, culori, subdomain de publicare
<code>site_versions</code>	Lanțul DCS: v0 cu cod complet (STORE), v1..N cu diff-uri (delta nodes)
<code>version_embeddings</code>	Vectorii de 1536 dimensiuni pentru fiecare instrucțiune, index HNSW

Tehnici de optimizare și integritate

- **Index HNSW** pe embeddings — accelerează căutarea celor mai apropiați vectori (timp logaritmic în loc de scanare liniară);
- **Ștergere în cascadă** — la ștergerea unui site, versiunile și embeddings-urile aferente se șterg automat, prevenind date orfane;
- **Funcție SQL dedicată** (`match_versions`) pentru căutarea semantică, apelată ca RPC;
- **Stocare pe delte** — economisește spațiu față de salvarea fiecărei versiuni ca fișier complet.

Modelul de acces la date

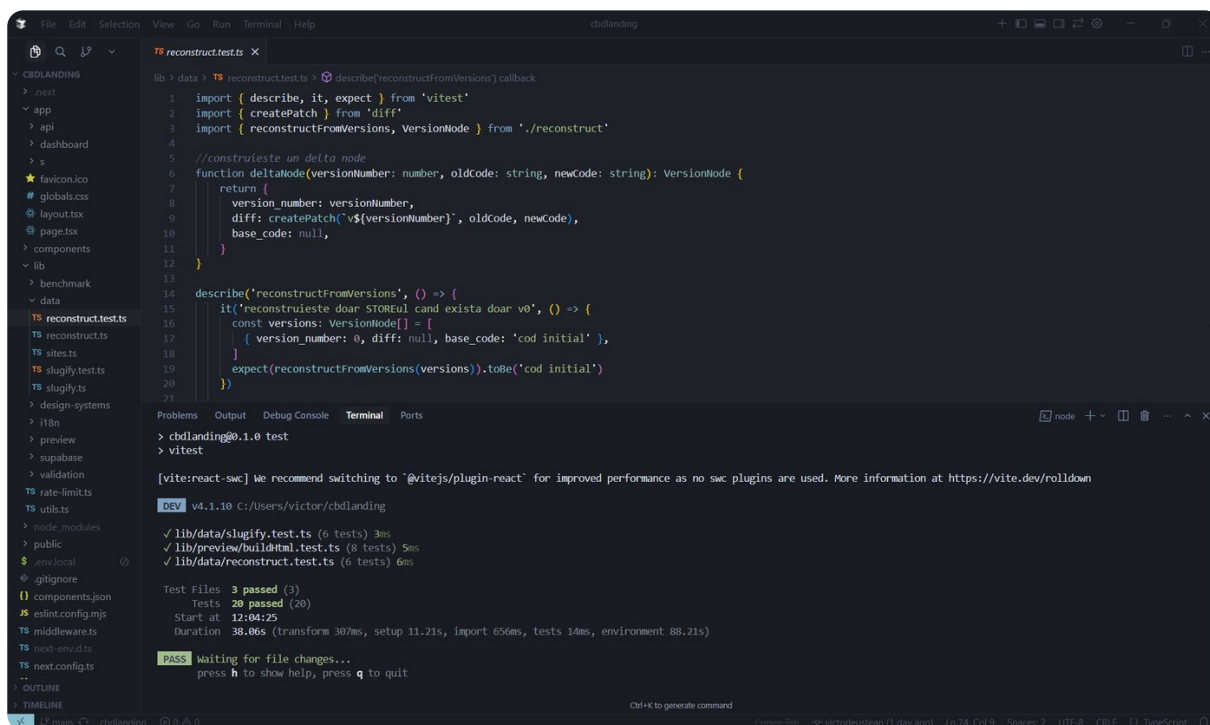
Accesul la bază se face exclusiv server-side, printr-un **data layer unic** în care fiecare funcție primește identificadorul utilizatorului (din Clerk) și filtrează strict după el. Aceasta concentrează verificarea de proprietate într-un singur loc — nu există interogare fără filtru de utilizator.

2.5 Testare

Logica critică a proiectului este acoperită de **teste unitare automate** scrise cu **Vitest**. Pentru a face testabilă reconstrucția DCS fără dependența de baza de date, logica pură de aplicare a diff-urilor a fost extrasă într-o funcție separată (`reconstructFromVersions`) — o decizie de arhitectură care separă logica de acces la date (*testability by design*).

Ce este testat

- **Reconstrucția din delte (DCS)**: reconstruirea corectă a STORE-ului, aplicarea unui diff, aplicarea unui lanț de diff-uri în ordine, reconstruirea unei versiuni intermediare, tratarea unui diff care nu se aplică (lanț rupt);
- **Curățarea codului generat** (`cleanGeneratedCode`): eliminarea import-urilor, transformarea iconițelor, escaparea etichetei de script;
- **Generarea de subdomenii** (`slugify`): diacritice, caractere speciale, limitarea lungimii, fallback.

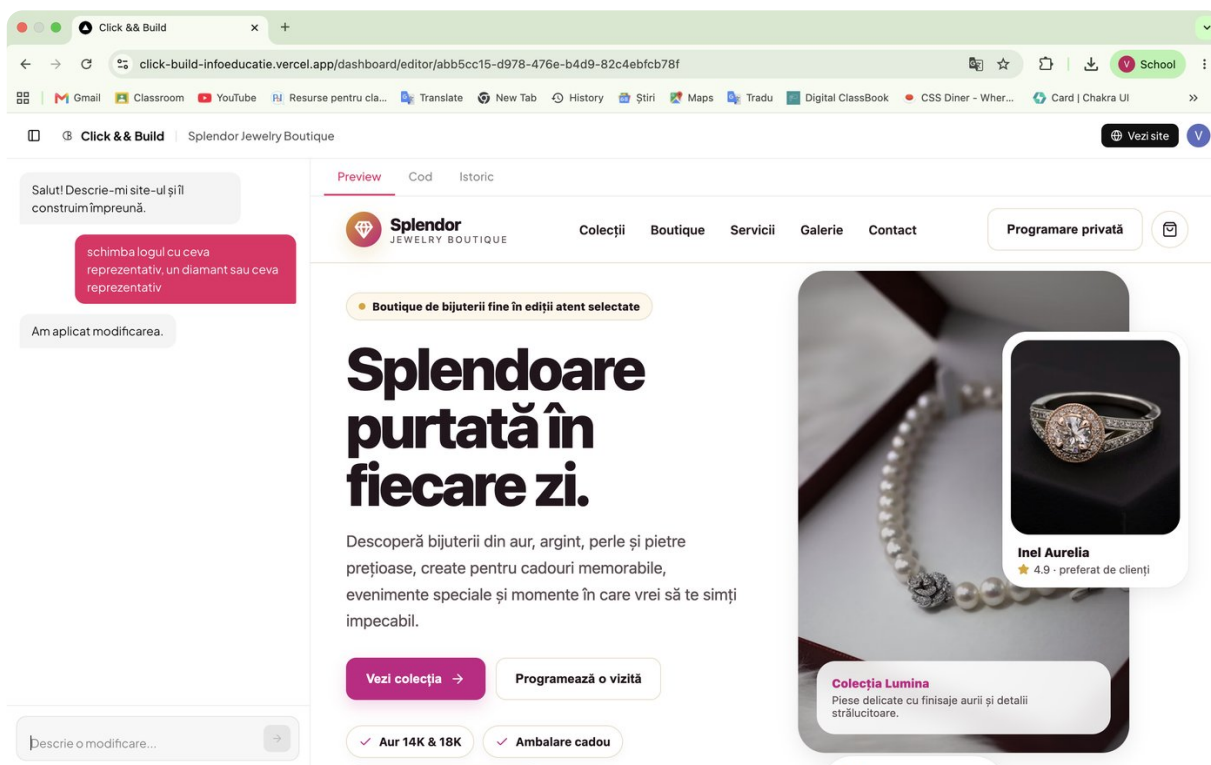


Rularea suitei de teste — 20 de teste trecute (3 fișiere), pe reconstrucția DCS, cleanGeneratedCode și slugify.

Tratarea cazului în care aplicarea unui diff eșuează (funcția aruncă eroare în loc să continue cu cod greșit) este un exemplu de **programare defensivă** verificată prin test — vezi și capitolul 6.

3. Funcționalități

Funcționalitate	Descriere
Generare din prompt	Site complet React dintr-o descriere în limbaj natural
Sugestii automate	Titlu, public țintă și paletă de culori propuse de AI
Editare conversațională	Modificări prin mesaje, cu context relevant injectat prin DCS
Preview live	Randare instant a site-ului generat în mediu izolat
Istoric versiuni	Timeline cu toate versiunile și instrucțiunile lor
Rollback semantic	Revenire la orice versiune, reconstruită din delte
Publicare	Site accesibil public la adresă proprie, fără cont
Management site-uri	Dashboard cu toate site-urile utilizatorului
Benchmark interactiv	Simulare live a economiei de tokeni DCS vs. abordarea clasică



Editorul în acțiune — panoul de chat în stânga, preview-ul site-ului generat în dreapta. O editare tocmai a fost aplicată.

Toate site-urile se salvează automat la generare. Utilizatorul își regăsește lucrările în dashboard, le poate redeschide pentru editare (codul curent fiind încărcat din baza de date) și le poate publica. Conținutul generat este realist și relevant domeniului — fără text de tip Lorem Ipsum.

4. Interacțiuni, design și accesibilitate

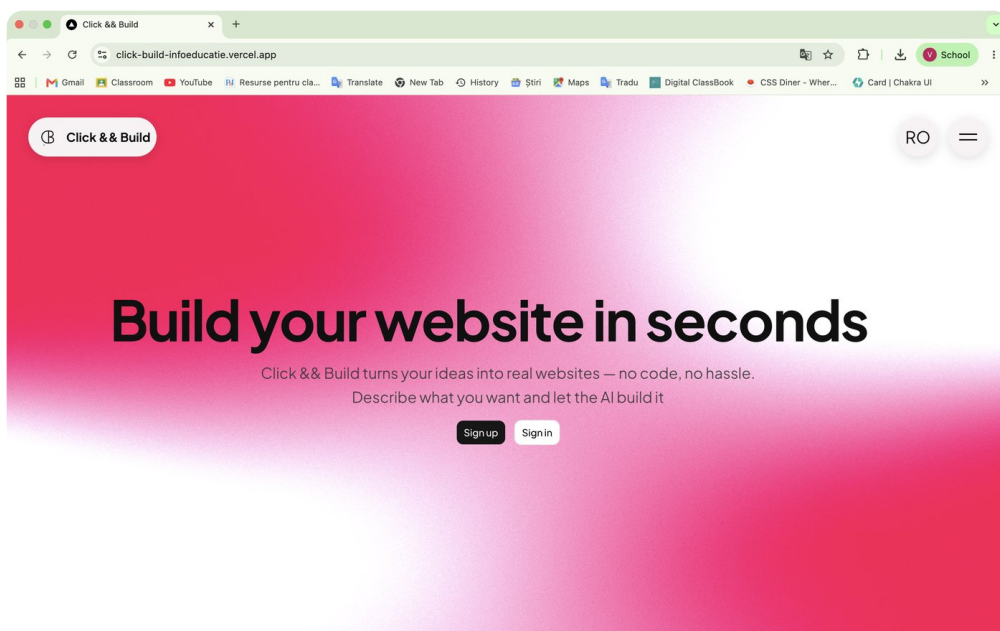
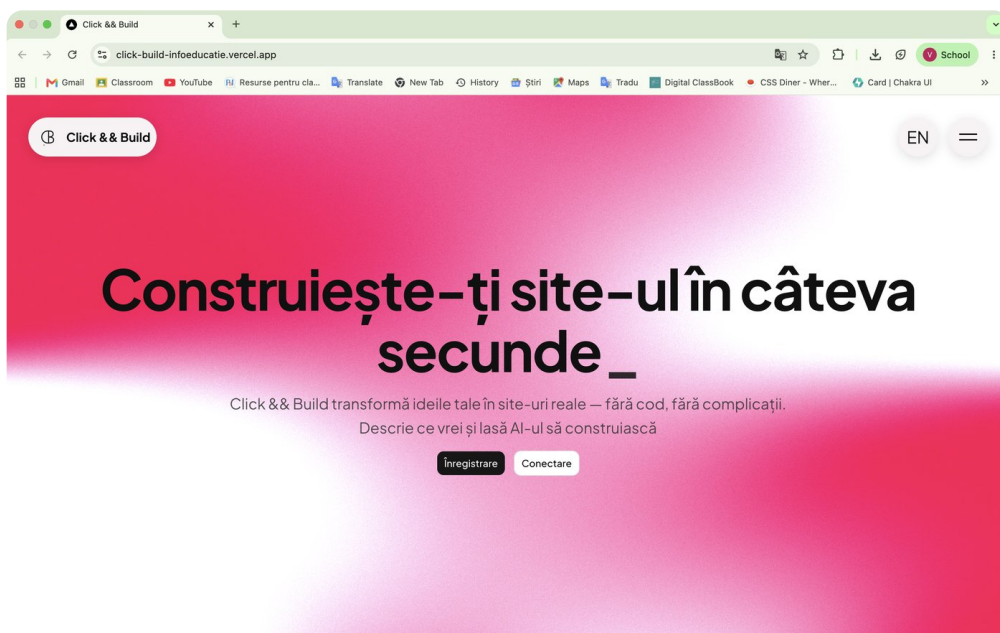
4.1 Interfața

Interfața urmează un principiu de claritate: un panou de chat în stânga pentru dialogul cu AI, un panou de preview în dreapta cu trei tab-uri (preview vizual, cod sursă, istoric versiuni). Designul este coerent — o paletă unitară, spacing consistent, animații subtile de feedback la interacțiuni.

4.2 Internaționalizare (RO / EN)

Interfața este complet **bilingvă**, română și engleză, printr-un sistem de internaționalizare **propriu** (fără servicii externe): un dicționar de traduceri, un context React și un hook `t()` care furnizează textul în limba activă. Un comutator cu steaguri permite schimbarea limbii instant, iar preferința este reținută între vizite.

Suplimentar, **site-urile generate** respectă limba promptului: dacă utilizatorul descrie site-ul în engleză, tot conținutul generat este în engleză; dacă în română, în română.



Aceeași pagină în română (sus) și engleză (jos) — interfața se comută instant din butonul cu steag.

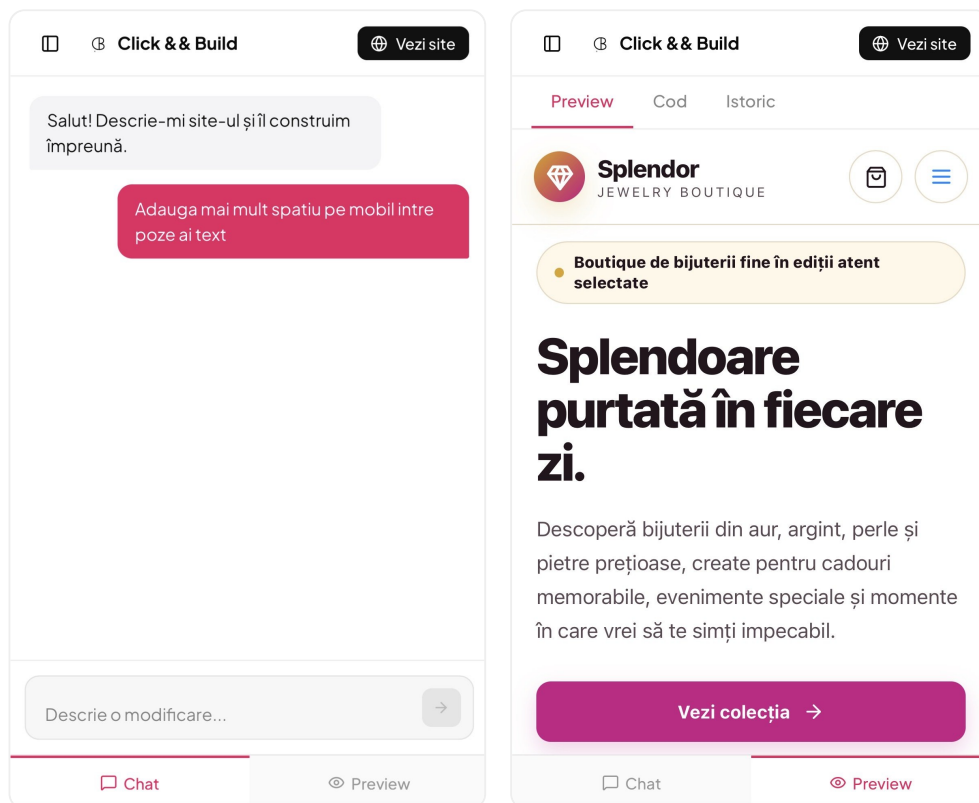
4.3 Accesibilitate

Interfața include tehnici de accesibilitate pentru utilizatori cu nevoi speciale:

- **Mod de contrast ridicat** — un comutator care intensifică contrastul text/fundal pentru utilizatorii cu deficiențe de vedere;
- **Respectarea `prefers-reduced-motion`** — animațiile sunt reduse automat pentru utilizatorii care au setată această preferință de sistem (rău de mișcare, sensibilitate vizuală);
- **Focus vizibil la navigarea cu tastatura** — toate elementele interactive au un indicator clar de focus, astfel încât aplicația să fie utilizabilă complet fără mouse.

4.4 Design responsiv

Interfața se adaptează diferitelor dimensiuni de ecran. Pe mobil, editorul comută între un mod „Chat” și unul „Preview” (fiecare pe tot ecranul), iar navbar-ul de pe landing se simplifică. Întrucât site-urile generate folosesc CSS inline (care nu suportă media queries), responsivitatea lor este asigurată printr-o tehnică de detectare a lățimii ecranului cu listener pe redimensionare — o regulă inclusă explicit în instrucțiunile de generare.



Editorul pe mobil — modul „Chat” (stânga) și modul „Preview” (dreapta), cu comutatorul de jos între ele.

5. Originalitate

Majoritatea generatoarelor de site-uri cu AI funcționează pe același principiu: un apel către un model, cu reîncărcarea întregului istoric la fiecare modificare. Click & Build se diferențiază prin elemente originale:

1. DCS — un sistem propriu de management al contextului

Spre deosebire de RAG-ul clasic (folosit pentru documente), DCS aplică retrieval-ul semantic la **versiuni de cod**, combinându-l cu stocare pe delte și reconstrucție prin diff-uri. Nu este un pattern preluat de pe raft, ci un sistem conceput pentru problema specifică a editării iterative. Dacă s-ar schimba furnizorul modelului AI, DCS ar rămâne — fiindcă este cod propriu, independent de furnizor.

2. Pipeline multi-model orchestrat

Coordonarea a mai multor modele specializate (cuvinte-cheie, design, generare cod) într-un flux unitar, cu imagini reale și design ca inspirație, produce rezultate superioare unui apel generic.

3. Vizibilitatea tehnologiei

DCS nu este o piesă ascunsă: timeline-ul de versiuni, rollback-ul semantic și benchmark-ul cantitativ fac tehnologia **vizibilă și verificabilă** pentru utilizator.

DCS transformă editarea iterativă cu AI dintr-un proces care devine tot mai scump și mai imprecis într-unul care rămâne ieftin și relevant, oricât de mult lucrezi la site.

6. Securitate

Securitatea a fost tratată pe mai multe niveluri, proporțional cu suprafața de risc a aplicației.

6.1 Izolarea codului generat (XSS)

Cel mai mare risc al aplicației este rularea de cod generat de AI. Acesta este randat într-un **iframe cu `sandbox="allow-scripts"`** — scripturile rulează (necesar ca React să funcționeze), dar **fără `allow-same-origin`**. În consecință, codul generat rulează într-o origine izolată și:

- nu poate accesa cookie-urile, sesiunea sau `localStorage`-ul aplicației gazdă;
- nu poate manipula DOM-ul paginii principale;
- nu poate efectua acțiuni în numele utilizatorului.

Aceasta este o protecție XSS reală: chiar dacă modelul ar genera cod malițios (inclusiv prin *prompt injection*), izolarea împiedică orice efect asupra aplicației sau utilizatorului. Poziția defensivă este matură — presupunem că generarea poate eșua și ne apărăm la nivelul execuției.

6.2 Validarea datelor de intrare

Toate rutele API validează datele primite cu scheme stricte (Zod) **înainte** de a le folosi: tipuri corecte, limite de lungime, format (identificatorii trebuie să fie UUID valide, culorile hex valide). Datele invalide sau malformate sunt respinse cu cod 400, prevenind injectarea de valori neașteptate.

6.3 Rate limiting (protecție DoS și abuz)

Rutele costisitoare (generare, editare, sugestii) sunt protejate prin limitarea numărului de cereri per utilizator într-o fereastră de timp. O cerere care depășește limita primește codul standard **429**. Aceasta previne atât abuzul de costuri (apeluri AI), cât și atacurile simple de tip denial-of-service.

6.4 Autentificare și control al accesului

- **Autentificare** prin Clerk — toate rutele sensibile verifică identitatea utilizatorului;

- **Filtrare per-utilizator** — data layer-ul impune ca fiecare interogare să fie filtrată după proprietarul resursei;
- **Acces exclusiv server-side** la baza de date — cheia privilegiată nu ajunge niciodată în browser;
- **Row Level Security** activat pe tabele pentru a bloca accesul anonim din client.

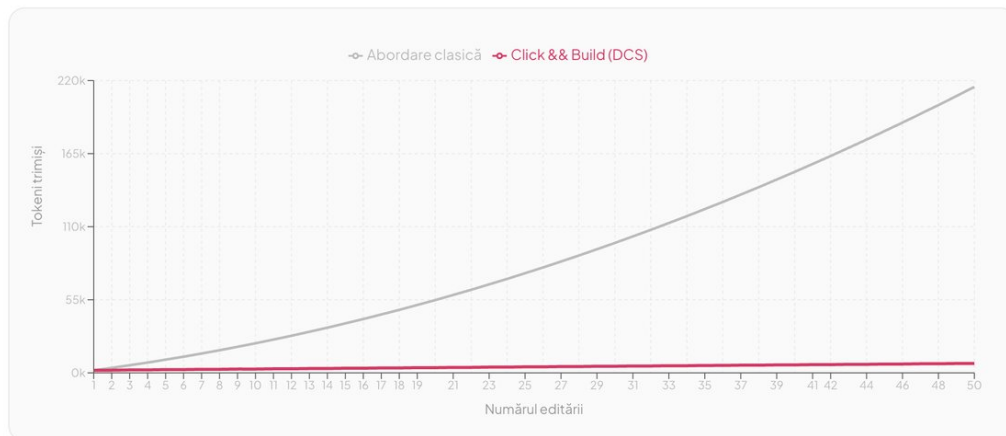
6.5 Programare defensivă

Apelurile externe (OpenAI, Unsplash, bază de date) sunt tratate defensiv: erorile sunt prinse și gestionate, salvarea este fail-soft (un eșec la o componentă secundară nu pierde munca utilizatorului), iar răspunsurile incomplete de la servicii externe sunt verificate înainte de utilizare. Reconstrucția DCS aruncă eroare explicită dacă un diff nu se aplică, în loc să continue cu cod corupt.

7. Performanță și validare cantitativă

7.1 Benchmark DCS vs. abordarea clasică

Pentru a demonstra cantitativ avantajul DCS, am implementat o simulare reproducibilă care compară, pe 50 de editări succesive ale aceluiași site, numărul de tokeni trimiși modelului de cele două abordări. Tokenii sunt numărați cu `tiktoken`. Simularea este integrată în aplicație și poate fi rulată live.



La a 50-a editare: **97% mai puțini tokeni** decât abordarea clasică.

Graficul benchmark-ului din aplicație — abordarea clasică (gri) crește exploziv, DCS (roz) rămâne aproape plat. La a 50-a editare: 97% mai puțini tokeni.

Editarea	Clasic (tokeni)	DCS (tokeni)	Economie
1	1.578	1.578	0%
5	7.885	1.871	76%
10	17.149	2.122	88%

20	40.418	2.737	93%
30	69.965	3.372	95%
50	147.836	4.627	97%

La prima editare cele două abordări sunt identice (nu există istoric). Diferența apare pe măsură ce conversația crește: abordarea clasică crește exploziv (reîncarcă tot istoricul), în timp ce DCS crește lent (doar codul curent se mărește). Către finalul celor 50 de editări, abordarea clasică **depășește fereastra de context tipică a modelelor (128k tokeni)** — deci pur și simplu nu mai funcționează — în timp ce DCS rămâne confortabil sub limită.

Rezultat: la a 50-a editare, DCS folosește cu **97% mai puțini tokeni** decât abordarea clasică (4.627 vs. 147.836).

7.2 Alte optimizări

- **Index HNSW** pentru căutarea vectorială rapidă;
- **Efort de raționament redus la editare** — reduce latența pe modificări simple;
- **Stocare pe delte** — minimizează spațiul ocupat de versiuni;
- **Salvare fail-soft** — embedding-ul (pas secundar) nu blochează salvarea codului.